# FEATURE AUDIT - AAVE-V3 `REPAY()`

Valya Zaitseva

February 2, 2026

## Table of Contents



## I. Feature Scope

1. The feature under the audit: `**repay()**`.

2. **Economic purpose:** it allows users to repay user debt (partially or totally) for themselves or on behalf of users, provided that they were explicitly delegated. The protocol can accept more assets than remaining debt at the call boundary, but must cap the effective repayment to outstanding debt and refund any excess.

3. **State modified:**

- protocol liquidity,
- total debt amount,
- user debt balances and borrowing configuration.

## II. Invariants / Design-Level Analysis

### Table 1. Grouped Invariants with Failure Surface and Threat Model

| State | Type | Invariant in plain English | Failure Surface (what can go wrong) | 5-lens questions | Threat Model / Attack | Enforced / Assumed | Severity / Impact | Test | Tested |
|---|---|---|---|---|---|---|---|---|---|
| Pre-state | STATE VALIDITY | repay() must revert if effective user debt is zero | Repay attempted on user debt == 0 | State: Can zero debt be repaid leaving dust or triggering events? | Attacker triggers a no-op, causing greifing, monitoring degradation | | Low / Low | Unit | No |
| | PROTOCOL-LEVEL SAFETY | reserve should not be pauzed or frozen | Reserve is pauzed or frozen | Economic/State: Can debt be paid if reserve is pauzed or frozen? | Attacker can bypass that reserve is pauzed or frozen and repay | | Medium / Low | Unit | No |
| | DEPENDENCY SAFETY | repay() must not rely on oracle values for correctness - e.g. oracle is set and price is not zero. | Oracle values are not correct | External: what happens if oracle returns zero or a stale price? | Attacker can manipulate an oracle to prevent repays or manipulate oracle-dependent health factor and liquidation. | | Medium / Medium | Unit | No |
| Mid-process | ATOMICITY | debt burn and liquidity increase must be atomic - no observable state where one happens without the other | Debt burn without liquidity change or liquidity decreases. | Economic: Can debt be burnt without liquidity increase? | Attacker can trigger `repay()`, burn debt but no actually repay, i.e. effective repay amount is not trasferred to the protocol | | High / High | Invariant test, fuzz | No |
| | AUTHORIZATION | only debtor or approved delegate can repay on behalf | Delegate can repay without approve | Authorization: Can a caller repay on behalf of another user without delegation approval? | Attacker repays for another person without permission -> manipulates HF -> avoids liquidation | | High / High | Unit | No |
| | ORDERING | Transfer asset before burn | Debt burned before transfer | Economic: Can debt be burned without transfer? | Attacker burns debt without actual asset transfer to the protocol | | High / High | Invariant test, fuzz | No |
| Post-state | USER ACCOUNTING | effective **debt** reduction <= user debt | Protocol reduces a user's debt by more than they owe | User accounting: Can the protocol forgive debt that never existed? | Attacker repays and protocol reduces debt by more than they owe due to rounding, index mismatch, or stale debt snapshot | | High / High | Invariant test, fuzz | No |
| | PROTOCOL SOLVENCY | **burned debt tokens** amount should be <= user debt | **Surplus burn** - debt tokens are burned in amount more that user's debt without accepting liquidity that covers extra burned debt tokens | Economic: Can amount of burned debt tokens be more that user debt? | Attacker **sends more assets t**o the protocol in order to burn more debt tokens than the user's debt, but protocol refunds assets difference and **burns more than debt tokens** amount because of rounding issues.<br>Attacker s**ends less assets tha**n debt, protocol **burns more than user's debt** because of rounding issues. | | High / High | Unit | No |
| | PROTOCOL SOLVENCY | **burned debt tokens** amount should be <= transferred repay amount to the protocol | **Surplus burn** - burned debt tokens amount is more than effective repay amount transferred to the | Economic: Can amount of burned debt tokens be more than effective repay amount transferred to the protocol? | Attacker sends less assets but debt is burned more than effective repay amount | | High / High | Invariant test, fuzz | No |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | | protocol | | | | | | |
| | PROTOCOL SOLVENCY | net reserve liquidity increase == effective repay amount | Liquidity changes does not correspond the actual assets amount transferred to the protocol | Economic: Can liquidity changes after repay differ from actually transferred amount of assets to the protocol? | Attacker sends minimum possible amount of assets to the protocol in order to change liquidity in the most meaningful way - either increase it or decrease - in both cases this change does not correspond to effective repay amount accepted by the protocol | | High / High | Invariant test, fuzz | No |
| | INDEX CONSISTENCY | indices may update, but must remain monotonic and consistent with accrued interest - **reserve indices are monotonic non-decreasing** and independent of individual user repay actions | indices decrease | Economic: Can indices be manipulated in order to reduce APR? | Attacker repays minimal amount that reduces indices -> repays other part of debt with decreased indices | | High / High | Invariant test, fuzz | No |
| | INDEX CONSISTENCY | a user's debt exposure is fully eliminated when scaled debt becomes zero, without mutating reserve indices | user still **has debt tokens even if debt becomes zero** and this leads to miscalculating of reserve indices | Economic: Can user still have debt tokens when his debt is already zero? | Attacker repays all debt but debt tokens are still on attacker's balance, mutating reserve indices | | Medium / High | Invariant test, fuzz | No |
| | DUST SAFETY | protocol must not create unrepayable or irreducible debt dust | unrepayable or irreducible debt dust stays in protocol, mutating indeces and internal accounting and fragmentating protocol liquidity? | Economic: Can user repay in a way some dust remains in protocol? | Attacker creates a lot of small debts, repays them in a way that some dust remains in protocol, leading to unrepayable liquidity amount is locked in protocol and will never be repaid and returned to the protocol effective liquidity. Indices will be unfluenced by this unpayable amount. | | High / High | Invariant test, fuzz | No |
| | USER ACCOUNTING | user health factor must never decrease as a result of 'repay()' | user health factor decreases as a result of repay. | Accounting: Can user's HF decrease as a result of repay? | Attacker may decrease some user's HF, especially if this attack is **combined with anauthorized repay** possibility. This will cause unfair user liquidation and attacker may liquidate attacked user and gain profit from this | | High / High | Invariant test, fuzz | No |
| | USER ACCOUNTING | user health factor must not improve without protocol liquidity increase | user health factor improves without protocol liquidity increase. | Accounting: Can user's HF improve as a result of repay without protocol liquidity increase? | Attacker may artificially improve their HF metric without transferring actual assets to the protocol, potentially avoiding liquidation unfairly. | | High / High | Invariant test, fuzz | No |
| | USER ACCOUNTING | user state is updated correctly on full repay | user state is updated wrongly on full repay | Accounting: Can user's state be updated incorrecly? | Attacker can manipulate user's state, especially if this attack is combined with anauthorized repay possibility. | | Medium / Medium | Invariant test, fuzz | No |

## III. Cross-Feature Invariant Consistency

**`repay()`** should reverse all borrow-induced state changes: decrease debt tokens and restore liquidity, should reverse principal effects, not necessarily index history.

## IV. Code Inspection

### 1. ‘repay()’ - Call Map and Structural Scan

```
repay()                                                    -> reduce what the user owes, pay value either by
    |                                                         burning aTokens or transfer underlying into the protocol
    |-> BorrowLogic.executeRepay()
        |-> VIEW: reserve.cache()                                          -> in-memory snapshots of addresses,
        |                                                                    indices and rates of a reserve
        |-> IRREVERSIBLE/STORAGE: reserve.updateState()                    -> updates indices and timestamp
        |-> VIEW: Helpers.getUserCurrentDebt                               -> gets user's stableDebt and
        |                                                                    variableDebt
        |-> ACCOUNTING BOUNDARIES/VIEW: ValidationLogic.validateRepay      -> validates repay
        |-> determine 'paybackAmount'
            |-> if InterestRateMode == STABLE -> paybackAmount = stableDebt
            |-> else -> paybackAmount = variableDebt
        |-> partial vs full repayment                                     -> reconsider 'paybackAmount' depending
        |                                                                    on effective repay amount
        |                                                                    transferred to protocol
        |-> if InterestRateMode == STABLE                                 |-> debt tokens are burned before
                                                                             transfer and update of interest rates
            |-> IRREVERSIBLE PIVOT/EXTERNAL: IStableDebtToken.burn()       |
        |-> else                                                           |
            |-> IRREVERSIBLE PIVOT/EXTERNAL: IVariableDebtToken.burn()     |
        |-> IRREVERSIBLE/STORAGE: reserve.updateInterestRates              -> updates interest rates and
        |                                                                    utilisation, affecting borrow and
        |                                                                    supply rates
        |-> IRREVERSIBLE/STORAGE: userConfig.setBorrowing                  -> user's borrowing flag is zeroed if
        |                                                                    he repaid all debt
        |-> IRREVERSIBLE/STORAGE: IsolationModeLogic.updateIsolatedDebtIfIsolated  -> adjust isolated debt if reserve is
        |                                                                    isolated
        |-> if useATokens == true
            |-> IRREVERSIBLE/STORAGE: IAToken.burn()                       -> no approve/transfer of underlying
            |                                                                token, burn aToken and reduce the
            |                                                                user's claim on reserve liquidity
        |-> else
            |-> IRREVERSIBLE/EXTERNAL: IERC20.safeTransferFrom()           -> transfer underlying into the
            |                                                                protocol
        |-> EMIT: Repay(asset, onBehalfOf, msg.sender, paybackAmount, useATokens)
```

### 2. ‘burn()’

#### a. stableDebt and variableDebt

-> oldIndex * interestSinceLastUpdated == getNormalizedVariableDebt() (@V:E aave here calls index as 'debt', i.e. getNormalizedVariableDebt should be getNormalizedVariableIndex)

---> oldIndex == reserve.variableBorrowIndex

---> interestSinceLastUpdated  == calculateCompoundInterest(rate, deltaTime)

-> variableDebtToken.super.balanceOf(user) == scaledDebt

variableDebt = variableDebtToken.super.balanceOf(user) * (oldIndex * interestSinceLastUpdated)

-> stableDebt calculation does not use stored 'oldIndex', it's index is always re-calculated based on per-user rate when current stableDebt is needed. This is possible because principal is stored for stableDebt is raw, not scaled (see below that it is scaled rarely)

-> index = calculateCompoundedInterest(userRate, deltaTime)

stableDebt = stableDebtToken.super.balanceOf(user) * index(per-user rate, deltaTime)

To sum up: stableDebt calculation does not use stored 'oldIndex', it's index (=growth factor) is always re-calculated when current stableDebt is needed (i.e. recomputed on demand from (userRate and now-lastUpdated)). This is possible because principal is stored for stableDebt as raw, not scaled by any index. There is no need to store scaled principal for stableDebt because the rate is individual for a user. But!! Stable rate may be rebalanced, i.e. changed. In this case stable principal will change, it will contain accrued interest now for the old per-user rate and new per-user rate will be applied to the new principal for future time. This rebalancing happens rare, comparing to variable rate.

But to culculate variableDebt we use stored scaled principal, which includes applied indices before the current time. It is necessary because variable index CHANGES OVER TIME frequently and should be applied on demand or once it is changed because there is no storage for variable rate in particular time range.

Both models of stable debt and variable debt are the same at the moment of global rate or per-user rate change. In this moment 'scaledBalance' (==scaledDebt) for variableDebt contains interest prior rate change. Same is for stableDebt -> the user principal updates to include the previous interest calculated for the old per-user rate. Scaled balance stays constant between updates, just like principal for stable debt.

Key takeaways: Operational difference:

Variable → global rate applied via global index

Stable → per-user rate applied on demand

### b. ‘IStableDebtToken.burn()’ - Call Map and Structural Scan

dsdsd

### c. ‘IVariableDebtToken.burn()’- Call Map and Structural Scan

dsdsdsd

## 3. Invariants Enforcement / Assumption Map

func -> enforced: invariant

assumed: invariant

V. Adversarial PoCs & Property Tests

3. Invariant 1 ..
4. Invariant 2 ..

VI. Report & Risk Advisory

Include all of the above plus severity, recommendations, and mitigation guidance